

REMEDIATION & ELEVATION ROADMAP

Veridex Enterprise Platform

From v1.0.0 audit findings to a production-ready, enterprise-grade release

Source input	50-item audit — frontend, backend, security, infrastructure
Added scope	27 additional items identified during planning (tagged NEW throughout)
Total scope	77 tracked items across 7 phases
Structure	Phase 0 → Phase 6, sequenced by dependency, blast radius, and compounding risk
Indicative timeline	16 weeks for a small platform team (4–6 engineers); compress or stretch per team size
Output	This document — plan only. No code has been written or changed.

HOW TO READ THIS DOCUMENT

Every item below is tagged with its Source: AUDIT for the 50 original findings (kept in their original numbering, e.g. #1–#50) or NEW for items added during this planning pass. Priority (P0–P3) reflects risk and blast radius, not difficulty. Effort is a rough sizing (S / M / L / XL) for a mid-sized team, meant for sequencing, not estimation. Each phase opens with a short rationale for why its items are grouped and ordered as they are.

Contents

Executive Summary

The audit correctly identifies 50 real gaps between Veridex v1.0.0 and a production-ready enterprise platform. Taken as a flat list, though, it reads as equally urgent everywhere — a missing favicon (#48) sits next to a hardcoded auth bypass (#4) with no signal about which one gets someone's credentials stolen. This roadmap re-sequences all 50 items by actual risk and dependency, and adds 27 items the audit didn't cover, most of them because "list of bugs" and "production readiness plan" are different documents — the second one needs observability, secrets architecture, DR targets, and a rollback story that a bug list has no natural place for.

Two sequencing principles drove every ordering decision:

- **Fix the thing that removes trust first.** An auth bypass, a plaintext-secrets pattern, or an unbounded query aren't roadmap items to schedule alongside UI polish — they're incidents waiting to happen and they gate everything else. Phase 0 exists to close these before any other work starts, including work on this same list.
- **Don't build the feature before the floor under it.** Chat history persistence (#9, #21) is pointless without a backend model to persist it to. MFA (#35) is premature without a refresh-token flow (#33) to attach it to. CD (#49) is dangerous without E2E tests (#50) to gate it. Every phase transition below exists because of a concrete dependency like this, not just topic grouping.

The plan is organized into seven phases. Phase 0 is a one-week security stop-gap. Phases 1–4 systematically close the audit's 50 items. Phase 5 is where the 27 added items live — the elevation work that turns "bugs fixed" into "elite." Phase 6 is launch readiness: the gate that confirms everything actually holds together before a GA tag goes out.

Phase 0 — Stop the Bleeding	5 items · Week 1 · Security-critical, ships before anything else
Phase 1 — Foundation Stabilization	11 items · Weeks 2–3 · Error handling, validation, data integrity floor
Phase 2 — Core Feature Completion	13 items · Weeks 4–6 · Make the mocked features real
Phase 3 — Security & Compliance Hardening	12 items · Weeks 7–8 · MFA, audit logs, CSRF, retention, ToS
Phase 4 — Infrastructure & Observability	13 items · Weeks 9–10 · Docker, Helm, backups, centralized logging
Phase 5 — Elite Additions	27 new items · Weeks 11–14 · Tracing, DR, multi-tenancy, a11y, FinOps, feature flags
Phase 6 — Launch Readiness	7 items · Weeks 15–16 · E2E coverage, CD pipeline, staged rollout, go/no-go

How the Phases Depend on Each Other

Before the phase-by-phase detail, it's worth being explicit about why the order is what it is — because a few of the audit's 50 items would be actively counterproductive to build early. This diagram shows the load-bearing dependencies: an arrow means "the item at the tail makes the item at the head safe or possible," not just "comes before it chronologically."

WIDGET

A dependency diagram is rendered inline below as an interactive visual — see the diagram immediately following this note in the live conversation. It is not reproduced as a static image in this Word document; the phase tables in this document carry the same sequencing information in tabular form.

The three dependency chains that shaped this plan

- **Auth chain:** kill the dev bypass (P0) → build refresh tokens (P3) → layer on MFA (P3) → only then does RBAC/OPA verification (P2) mean anything, since a role check is worthless if the identity behind it can be forged or never expires.
- **Data chain:** add pagination (P2) → then add delete/update endpoints (P2) → then add retention TTLs (P3) — you don't want to build automatic deletion logic against tables that still return unbounded result sets to every caller.
- **Ship chain:** error boundaries + global exception handling (P1) → E2E tests (P6) → CD pipeline (P6) — automating deployment before you have tests that would catch a bad deploy just automates the outage.

Phase 0 — Stop the Bleeding

WEEK 1 • SECURITY-CRITICAL • SHIPS BEFORE ALL OTHER WORK

These five items are not "the top of the backlog" — they're pre-conditions for doing any other work safely. Every one of them is either an active vulnerability in v1.0.0 or a landmine that a well-meaning contributor could trip during Phase 1–2 work. Nothing else starts until this phase is done, including work by parallel workstreams, because Phase 1 changes touch the same auth and API surface these items harden.

WHY THIS PHASE EXISTS ON ITS OWN

The audit lists #4 (hardcoded auth bypass) as item 4 of 50 — same visual weight as a missing favicon. That's the single most important resequencing decision in this document. A `// LOCAL DEV BYPASS` comment that compiles into production isn't a backlog item; it's a live vulnerability sitting in main right now.

The other four items here (#18, #25, #30, #42) share a property: each is a one-line-of-config fix with a disproportionate blast radius, and each is trivially exploitable by anyone who finds it before the team does.

#	Item	Source	Pri.	Effort	Track
1	Remove or hard-gate the AuthContext dev bypass behind an env check that fails the build in prod	#4	PO	S	Frontend / Auth
2	Lock CORS to the explicit production frontend origin — remove wildcard	#18	PO	S	Backend
3	Parameterize all dynamic Cypher queries in the Knowledge Graph connector	#25	PO	M	Backend / Data
4	Replace hardcoded `localhost:8000` API fallback with required env var + startup fail-fast	#30	PO	S	Frontend / Config
5	Move Postgres/Neo4j credentials out of docker-compose.prod.yml into .env / secret store	#42	PO	S	Infra

Exit criterion for Phase 0: a fresh production build with debug flags off cannot be logged into without real credentials, cannot execute an injected Cypher payload through the Knowledge Graph, cannot silently fall back to a broken localhost API, and no secret is readable from a checked-in compose file.

Phase 1 — Foundation Stabilization

WEEKS 2–3 · RELIABILITY FLOOR BEFORE FEATURE WORK

Phase 1 is about the difference between "the feature is missing" and "the app falls over." Nothing here adds a feature — it's error handling, validation, and query-safety work that makes Phase 2's feature-building safe to do without every new feature inheriting the same crash-on-error and unbounded-query patterns.

WHY THIS ORDER

Global error boundary and exception handling (#2, #28) come first because every subsequent phase writes new code paths that can throw. Building those paths before the safety net exists means retrofitting error handling into six phases of new code instead of inheriting it once.

Pagination (#16) is grouped here rather than with the API completeness work in Phase 2, because it changes the response shape of existing endpoints. Better to land that shape change before Phase 2 builds new UI against these endpoints, not after.

#	Item	Source	Pri.	Effort	Track
6	Add Next.js error.tsx global error boundary	#2	P1	S	Frontend
7	Add FastAPI global exception handler with standardized JSON error schema	#28	P1	M	Backend
8	Add loading.tsx suspense states for slow page transitions	#3	P1	S	Frontend
9	Add custom not-found.tsx (404) page	#1	P2	S	Frontend
10	Add pagination (skip/limit) to /telemetry, /tasks, and other list endpoints	#16	P1	M	Backend
11	Add strict Qdrant vector-dimension validation before insert	#26	P1	S	Backend / Data
12	Add react-hook-form + zod validation to /login and /register	#5	P1	M	Frontend
13	Add password requirement UI (length/complexity) on register	#6	P2	S	Frontend
14	Configure Celery max_retries + exponential backoff for external API calls	#24	P1	M	Backend
15	Add API rate limiting (slowapi) to login and LLM-heavy routes	#17	P0	M	Backend
16	Add JWT expiration pre-check in AuthContext before firing requests	#32	P1	S	Frontend / Auth

Note on #17: the audit places rate limiting in the "Backend APIs" section (16–30), but it is functionally a security control — brute-forcing the login endpoint is a direct account-takeover vector. It's sequenced early here (P0 priority) despite living in the Phase 1 batch, because it depends on nothing else in this phase and there's no reason to defer it.

Phase 2 — Core Feature Completion

WEEKS 4–6 · TURN MOCKED FEATURES INTO REAL ONES

This is the largest phase in the original audit and the one users will feel most directly — five separate features currently render hardcoded or mock data as if it were live. Each of these needs a backend counterpart built first (or alongside), which is why this phase pairs frontend items with their backend prerequisite in the same row rather than splitting them across phases.

WHY THIS ORDER

Chat history (#9, #21) is sequenced first because it's the highest-visibility mock — every demo of the product hits the chat page, and "refresh loses your conversation" is the fastest way to lose enterprise buyer confidence in a pilot.

Connectors (#11, #12, #20) come as a group because 'Add' doing nothing (#12) and no update endpoint (#20) are really one feature — a config modal — split into three audit line items. Building them separately would mean building the modal twice.

Evaluations (#13, #22) and Resilience/Chaos (#14) are lower urgency because they're admin/ops-facing rather than end-user-facing, but they need the same real-data treatment before Phase 5's observability work has anything real to observe.

#	Item	Source	Pri.	Effort	Track
17	Add SQLAlchemy model + endpoints to persist chat history by thread_id	#21	P1	L	Backend
18	Wire /chat page to persisted history instead of local React state	#9	P1	M	Frontend
19	Add auto-expanding multi-line chat input	#10	P3	S	Frontend
20	Add PUT/PATCH endpoints for Data Connectors	#20	P2	M	Backend
21	Build "Add New Source" configuration modal + onClick wiring	#12	P2	M	Frontend
22	Replace hardcoded connector array with live fetch from backend API	#11	P2	S	Frontend
23	Persist evaluation run results to Postgres for trend analysis	#22	P2	M	Backend
24	Wire /evaluations page to real historical runs instead of mock array	#13	P2	S	Frontend
25	Replace setTimeout chaos-log simulation with real SSE stream from runChaosTest	#14	P2	M	Full-stack
26	Add zoom/pan/click-to-inspect on Knowledge Graph (replace static SVG)	#8	P2	L	Frontend
27	Add delete/archive endpoints for tasks, connectors, telemetry logs	#19	P2	M	Backend
28	Add "Forgot Password" flow to login screen	#7	P2	M	Full-stack
29	Add mobile-responsive sidebar (hamburger nav) and fix grid breakpoints	#15	P2	M	Frontend

Phase 3 — Security & Compliance Hardening

WEEKS 7–8 · FROM "PATCHED" TO "DEFENSIBLE"

Phase 0 stopped the active bleeding. Phase 3 is the deeper security work that an enterprise buyer's security questionnaire will actually ask about: MFA, audit trails, encryption at rest, and the compliance paperwork the audit correctly flags but doesn't design.

WHY THIS ORDER

Refresh tokens (#33) come before MFA (#35) deliberately — MFA bolted onto a single long-lived access token doesn't meaningfully improve session security, since the token that survives a device compromise doesn't care whether a second factor was checked at login.

RBAC token verification (#23) is placed after refresh tokens for the same reason: it's pointless to harden how carefully a role claim is checked in a JWT whose issuance and expiry story isn't solid yet.

#	Item	Source	Pri.	Effort	Track
30	Implement short-lived access token + HttpOnly refresh token flow	#33	P0	L	Backend / Auth
31	Verify RBAC role claims are actually checked by OPA against JWT, not just injected client-side	#23	P0	M	Backend
32	Add TOTP-based MFA/2FA as a requirement for enterprise accounts	#35	P1	L	Full-stack
33	Encrypt stored GitHub/Slack API keys at rest (Fernet or equivalent)	#31	P0	M	Backend
34	Add audit log table for successful/failed login attempts	#36	P1	M	Backend
35	Add OPA policy result caching (Redis) to remove per-request latency	#34	P2	M	Backend / Infra
36	Add data retention TTL / purge job for telemetry logs older than 90 days	#37	P2	M	Backend / Data
37	Add mandatory ToS / Privacy Policy checkboxes to /register (GDPR/CCPA)	#38	P1	S	Frontend / Legal
38	Enforce npm audit and Python safety scans as failing CI checks	#39	P1	S	DevOps
39	Add CSRF token support (pre-emptive, ahead of any move to cookie auth)	#40	P3	S	Backend
40	Add developer-facing /docs (Swagger UI) link in the frontend footer	#29	P3	S	Frontend
41	Security review pass: re-run Phase 0 checks against all Phase 1–3 code	NEW	P1	M	Security

Item 41 is new: a checkpoint, not a feature. After three phases of active development on the auth and API surface, it's worth explicitly re-verifying the five Phase 0 fixes are still intact — a rebase or a well-meaning refactor can silently reintroduce a wildcard CORS origin or an unparameterized query.

Phase 4 — Infrastructure & Observability

WEEKS 9–10 · MAKE PRODUCTION DEPLOYMENTS BORING

This phase closes the audit's infrastructure section (#41–#50, minus the two testing/CD items reserved for Phase 6) and starts laying the observability groundwork that Phase 5's additions build on.

#	Item	Source	Pri.	Effort	Track
42	Add healthcheck blocks to frontend + backend containers in docker-compose.prod.yml	#41	P1	S	Infra
43	Add SSL/TLS termination via Nginx / Traefik / Istio Ingress	#43	P0	M	Infra
44	Set explicit CPU/memory requests + limits in helm/veridex charts	#44	P1	M	Infra
45	Set up centralized logging (Loki or ELK) across Celery, FastAPI, Next.js, OPA	#45	P1	L	Infra
46	Automate database backups (pg_dump + Neo4j backup) on a schedule	#46	P0	M	Infra
47	Set output: 'standalone' in next.config.js to shrink production image size	#47	P3	S	Frontend / Infra
48	Replace default Next.js metadata/favicon with Veridex branding	#48	P3	S	Frontend
49	Add WebSocket support for the interactive chat agent (replace one-way SSE)	#27	P2	L	Full-stack
50	Verify backup restore procedure actually works (not just that backups run)	NEW	P0	M	Infra
51	Add structured JSON logging with correlation/request IDs across services	NEW	P1	M	Backend / Infra
52	Add infra-as-code drift detection (Terraform/Helm diff in CI)	NEW	P2	M	Infra
53	Add a staging environment that mirrors prod topology for pre-release testing	NEW	P1	L	Infra
54	Document and rehearse a rollback procedure for a bad deploy	NEW	P1	S	Infra / Docs

Item 50 matters more than it looks: #46 in the original audit asks for backups to exist, but a backup that has never been restored is a hypothesis, not a safety net. This is the single most common gap between "we have backups" and "we recovered from an incident."

Phase 5 — Elite Additions

WEEKS 11–14 · THE GAP BETWEEN "BUGS FIXED" AND "ELITE"

This is where most of the added scope lives. The 50-item audit is a solid bug-and-gap list for an application; it is not a checklist for an enterprise platform a security-conscious buyer will actually trust with their code and data. The nine categories below are the ones a platform at this stage is missing entirely — not degraded versions of something in the audit, but categories with zero representation in the original 50.

5.1 — Observability Beyond Logging

The audit's #45 gets logs into one place. That answers "what happened." It doesn't answer "is anything wrong right now" or "why is this request slow." Those need metrics, tracing, and SLOs — three different tools doing three different jobs.

#	Item	Source	Pri.	Effort	Track
55	Add distributed tracing (OpenTelemetry) across Next.js → FastAPI → Celery → Neo4j/Qdrant	NEW	P1	L	Infra / Backend
56	Add Prometheus metrics + Grafana dashboards for API latency, error rate, queue depth	NEW	P1	L	Infra
57	Define SLOs for the chat and knowledge-graph endpoints and alert on burn rate	NEW	P2	M	Infra / Product
58	Add synthetic uptime checks for the public-facing login and chat flows	NEW	P2	S	Infra

5.2 — Secrets Management Architecture

The audit flags plaintext secrets twice (#31, #42) and both fixes are correct but tactical — encrypt this field, move that password to .env. Neither proposes the actual architecture: a single secrets manager that issues short-lived, auditable credentials, so encryption keys, database passwords, and third-party API tokens aren't each solved with a different one-off pattern.

#	Item	Source	Pri.	Effort	Track
59	Adopt a secrets manager (Vault, AWS Secrets Manager, or equivalent) for all credentials	NEW	P1	L	Infra / Security
60	Migrate the Fernet encryption key itself (from #31) out of app config into the secrets manager	NEW	P1	M	Backend / Security
61	Add automated secret rotation for database and third-party API credentials	NEW	P2	M	Infra / Security

5.3 — Multi-Tenant Data Isolation

Nothing in the 50-item audit addresses whether one enterprise customer's data can leak into another's. For a platform storing customer code, chat transcripts, and Knowledge Graph entities, this isn't optional scope — it's the difference between selling to one company and selling to many.

#	Item	Source	Pri.	Effort	Track
---	------	--------	------	--------	-------

#	Item	Source	Pri.	Effort	Track
62	Add tenant_id scoping to every Postgres query, Neo4j Cypher query, and Qdrant filter	NEW	P0	XL	Backend / Data
63	Add automated tests that assert cross-tenant queries return zero rows	NEW	P0	M	Backend / QA
64	Document tenant isolation model for the enterprise security questionnaire	NEW	P2	S	Security / Docs

5.4 — Disaster Recovery Targets

#46 gets backups running. It doesn't say how much data the business is willing to lose (RPO) or how long it's willing to be down (RTO) — numbers that should drive the backup frequency, not the other way around.

#	Item	Source	Pri.	Effort	Track
65	Define RTO/RPO targets per data store (Postgres, Neo4j, Qdrant) with business sign-off	NEW	P1	S	Infra / Leadership
66	Build and test a full-region failover runbook against those targets	NEW	P2	L	Infra

5.5 — Accessibility (a11y)

Zero mentions across all 50 audit items. For an enterprise buyer, WCAG conformance is frequently a procurement requirement, not a nice-to-have — its absence here is the single biggest blind spot in the original list.

#	Item	Source	Pri.	Effort	Track
67	Run automated WCAG 2.1 AA audit (axe-core) across all pages in CI	NEW	P1	M	Frontend / QA
68	Fix keyboard navigation and focus management on Knowledge Graph and Chat	NEW	P1	M	Frontend
69	Add proper ARIA labeling to the chat interface and evaluation dashboards	NEW	P2	M	Frontend

5.6 — LLM Cost Guardrails

#24 adds retry logic for flaky OpenAI calls — correct, but retries without a spend ceiling on an LLM-heavy product is a blank check. A retry storm during an upstream outage can turn a reliability fix into a cost incident.

#	Item	Source	Pri.	Effort	Track
70	Add per-tenant and global LLM spend caps with circuit-breaker behavior	NEW	P1	M	Backend
71	Add token-usage dashboards broken out by tenant, route, and model	NEW	P2	M	Backend / Infra

5.7 — API Versioning Policy

The hardcoded `/api/v1` fallback (#30, fixed in Phase 0) implies a version strategy exists. It doesn't — there's no policy for what changes require a v2, how long v1 stays supported, or how the frontend signals which version it expects.

#	Item	Source	Pri.	Effort	Track
72	Write and publish an API versioning + deprecation policy	NEW	P2	S	Backend / Docs
73	Add version negotiation via Accept header or URL path, enforced in FastAPI routing	NEW	P3	M	Backend

5.8 — Feature Flagging

Not in the original 50, but a direct prerequisite for Phase 6's CD pipeline (#49) to be usable safely — without flags, every deploy is an all-or-nothing exposure, which defeats the point of automating deployment in the first place.

#	Item	Source	Pri.	Effort	Track
74	Add a feature flag service (LaunchDarkly, Unleash, or a lightweight in-house flag table)	NEW	P2	M	Backend / Infra
75	Gate the WebSocket chat migration (#27/#49) and Knowledge Graph rewrite (#8) behind flags	NEW	P2	S	Full-stack

5.9 — Data Export & "Right to Be Forgotten"

#38 adds ToS/Privacy checkboxes at signup — the legal consent side. It doesn't build the operational side: an actual mechanism for a user or tenant to export their data or request deletion, which GDPR/CCPA compliance requires beyond the checkbox.

#	Item	Source	Pri.	Effort	Track
76	Build a user/tenant data export endpoint (JSON/CSV bundle of all owned records)	NEW	P2	M	Backend
77	Build a verified data-deletion workflow across Postgres, Neo4j, and Qdrant	NEW	P2	L	Backend / Data

Phase 6 — Launch Readiness

WEEKS 15–16 · THE GATE BEFORE A GA TAG

The audit's two testing/deployment items (#49, #50) are saved for last on purpose. A CD pipeline without E2E coverage automates shipping bugs faster; E2E coverage without a CD pipeline is a manual safety net nobody scales. They need to land together, and only after every prior phase has given them something real to test and something safe to deploy.

#	Item	Source	Pri.	Effort	Track
78	Add Playwright E2E tests covering login, chat, connectors, and knowledge graph flows	#50	P0	L	QA / Full-stack
79	Build CD pipeline: auto build + push Docker images on tag release	#49	P1	M	DevOps
80	Wire E2E suite as a required, blocking CD gate (no green tests, no deploy)	NEW	P0	S	DevOps / QA
81	Add canary or blue-green rollout strategy using the Phase 5 feature-flag layer	NEW	P1	M	Infra
82	Run a load test against production-scale data volumes (millions of telemetry rows)	NEW	P1	M	Infra / QA
83	Run a full incident-response tabletop exercise using the Phase 5 DR runbook	NEW	P2	S	Cross-functional
84	Final go/no-go review against this document's exit criteria for every phase	NEW	P0	S	Leadership

Exit criterion for Phase 6, and for this roadmap as a whole: a tagged release can go from git tag to production through the CD pipeline with zero manual steps, is blocked automatically if E2E tests fail, can be rolled back inside minutes using the Phase 4 rollback runbook, and has been load-tested at a data volume representative of a real enterprise deployment rather than the demo dataset.

Quick Reference — Full Index by Priority

For teams that want to slice the same 84 items by urgency rather than by phase — useful for a first sprint planning session where P0 items across every phase need pulling forward regardless of their "native" phase.

P0 — Critical (ships first, regardless of phase)

- Remove AuthContext dev bypass (#4) · Lock CORS (#18) · Parameterize Cypher queries (#25) · Fix hardcoded API fallback (#30) · Move Docker secrets to .env (#42)
- Add API rate limiting on login/LLM routes (#17)
- Refresh token flow (#33) · Verify RBAC/OPA claim checking (#23) · Encrypt API keys at rest (#31)
- SSL/TLS termination (#43) · Automated + verified-restorable backups (#46, NEW-50)
- Tenant data isolation + cross-tenant test coverage (NEW-62, NEW-63)
- E2E test coverage (#50) · E2E as blocking CD gate (NEW) · Final go/no-go review (NEW)

P1 — High

- Global error boundary + exception handling (#2, #28) · Pagination (#16) · Qdrant validation (#26)
- Form validation (#5) · Celery retry/backoff (#24) · JWT expiration pre-check (#32)
- Chat history persistence, backend + frontend (#21, #9) · MFA (#35) · Audit login logging (#36)
- ToS/Privacy checkboxes (#38) · Dependency vuln scanning in CI (#39) · Post-Phase-3 security re-check (NEW)
- Container healthchecks (#41) · Helm resource limits (#44) · Centralized logging (#45)
- Distributed tracing, metrics/dashboards (NEW) · Secrets manager adoption (NEW) · RTO/RPO targets (NEW)
- WCAG audit + keyboard nav fixes (NEW) · LLM spend caps (NEW) · Staging environment + rollback runbook (NEW)
- CD pipeline (#49) · Canary/blue-green rollout (NEW) · Production-scale load test (NEW)

P2 / P3 — Medium & Low

- Everything else in the phase tables above not listed as P0/P1 — largely feature completion (Phase 2), remaining infra polish (Phase 4), and the second half of each Phase 5 category (rotation, documentation, dashboards, deprecation policy).

Appendix — Legend & Planning Assumptions

Effort sizing

- **S (Small)**: under 2 days for one engineer.
- **M (Medium)**: 3 days to a week for one engineer, or 2–3 days paired.
- **L (Large)**: one to two weeks, often crossing frontend/backend or needing design input.
- **XL (Extra Large)**: multi-week, cross-cutting change touching most of the codebase — currently only tenant isolation (NEW-62) carries this size, which is accurate: retrofitting tenant scoping onto three different data stores after the fact is genuinely the largest single item in this plan.

Priority definitions

- **P0** Active vulnerability, data-loss risk, or blocking dependency for other P0 work. Fix before anything else in scope.
- **P1** Meaningfully increases risk or user-visible breakage if deferred past its phase. Should not slip a phase.
- **P2** Real gap, contained blast radius. Can shift a phase without compounding risk elsewhere.
- **P3** Polish, developer experience, or genuinely low-urgency scope. Safe to defer past this roadmap entirely if timeline pressure hits.

Assumptions behind the 16-week timeline

- A team of 4–6 engineers with full-stack coverage (not siloed frontend-only / backend-only), plus part-time DevOps and security support from Phase 3 onward.
- Phases run mostly sequentially as drawn, but within a phase, independent rows can and should run in parallel across the team — the ordering within a phase table generally reflects priority, not a required sequence, except where a row's item explicitly depends on the one above it.
- No item in Phase 5 or 6 is scoped to start before its stated week; several (tracing, feature flags, staging environment) are useful earlier and teams with spare capacity should pull them forward rather than treat every phase boundary as a hard gate.
- This document defines scope and sequence only. It intentionally contains no code, no config, and no architecture diagrams beyond the dependency model in the sequencing section — those are the natural next deliverables once the phase order here is agreed.